

Policy-based Broadcast Access Authorization for Flexible Data Sharing in Clouds

Hua Deng, Jixin Zhang, Zheng Qin, Qianhong Wu, Hui Yin, Aniello Castiglione

Abstract—Cloud storage services allow data owners to outsource their potentially sensitive data (e.g., private genome data) to remote cloud servers in a ciphertext form. To enable data owners to further share the data encrypted in ciphertexts, many proxy re-encryption (PRE) schemes are proposed. However, most schemes only support single-recipient or coarse-grained re-encryption, which may limit the flexibility for data sharing. To address this issue, we propose a Policy-based Broadcast Access Authorization (PBAA) scheme by introducing the well-established identity-based broadcast encryption (IBBE) and key-policy attribute-based encryption into PRE. In our PBAA scheme, a data owner can apply IBBE to encrypt his data to a group of recipients. More importantly, the data owner can generate a delegation key with an access policy, and send this key to the cloud such that it can convert any initial ciphertext satisfying the access policy into a new ciphertext for a new group of recipients. With these features, cloud users can share their remote data in a secure and flexible way. Security analysis and performance evaluation show that the PBAA scheme is secure and efficient, respectively.

Index Terms—Cloud computing, Data sharing, Proxy re-encryption, Broadcast encryption.

1 INTRODUCTION

CLOUD computing has been widely accepted as a solution for data maintenance due to its expandability and on-demand deployment characteristics [1], [2]. With the increasing popularity of cloud computing, more and more personal record management systems are utilizing cloud computing platform to collect, store and share data. For instance, personal health records are outsourced to or provided by third-party cloud service providers such as Microsoft Health Vault [3] and Google Health [4], which makes sharing of the health information among different medical institutions or individuals more convenient and efficient.

Despite of the conveniences and benefits brought by cloud computing, there exist serious concerns about the privacy of data in cloud computing. Once the data has been outsourced to the cloud, the data owner loses physical access control on the data; the data owner may worry about whether his private data has been leaked by the cloud. The conventional solution is to encrypt the data before outsourcing it to the cloud. Such a solution, however,

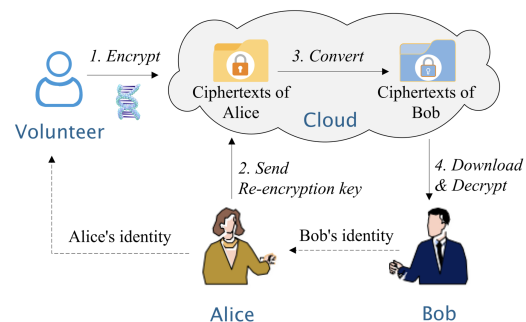


Fig. 1: IBPRE for Cloud Data Sharing

brings difficulty for sharing data with different users. Obviously, the data owner can first download the ciphertext from the cloud and decrypt it using his private key, and then encrypt the data to different recipients respectively. This straightforward method, nevertheless, is too complicated and inefficient.

Proxy re-encryption (PRE [5]) could be useful to address the dilemma for sharing encrypted data. In PRE, a proxy (e.g., the cloud) with some necessary information (a.k.a. re-encryption key) can transform a ciphertext intended for Alice into a new ciphertext for Bob. Identity-based PRE (IBPRE [6]) inherits all the merits of PRE and additionally allows any recognizable strings to serve as public keys, which alleviates the public-key management burden in traditional PRE systems. Due to its attractive features, IBPRE has been widely used in many practical applications such as cloud data sharing [6], [7], [8], e-mail forwarding [9], online social networks [10] and micro-video subscribing systems [11].

Fig. 1 illustrates an application of IBPRE in cloud data sharing. Suppose that there are volunteers willing to contribute their personal genome data for medical research purpose. The cloud is utilized to collect and share the genome data. To protect data from being accessed by unauthorized users or the cloud, a volunteer encrypts his data before outsourcing. With IBPRE, the volunteer

Corresponding authors: Zheng Qin; Qianhong Wu.

- H. Deng is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, China; with State Key Laboratory of Cryptology, P. O. Box 5159, Beijing, 100878, China; and also with Hangzhou Innovation Institute, Beihang University, Hangzhou, 310052, China. E-mail: hdeng@hnu.edu.cn
- J. Zhang is with the School of Computer Science, Hubei University of Technology, Wuhan, 430068, China. E-mail: zhangjixin@hnu.edu.cn
- Z. Qin is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, China. E-mail: zqin@hnu.edu.cn
- Q. Wu is with the School of Cyber Science and Technology, Beihang University, Beijing, 100191, China. E-mail: qhwu@xidian.edu.cn
- H. Yin is with the College of Applied Mathematics and Computer Engineering, Changsha University, Changsha, 410022, China. E-mail: yhui@ccsu.edu.cn
- A. Castiglione is with the Computer Science Department of the University of Salerno, Fisciano (SA)-I-84084, Italy. E-mail: castiglione@ieee.org

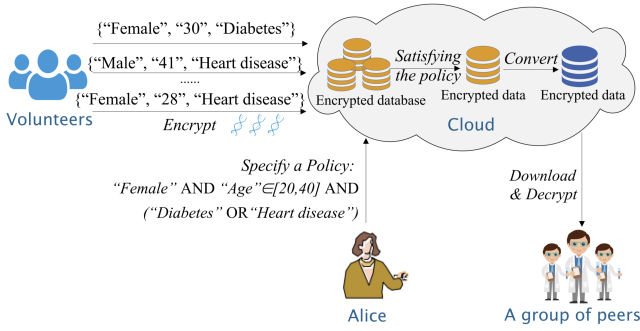


Fig. 2: Flexible Data Sharing in the Cloud

uses the identity of the researcher Alice to encrypt the genome data such that only Alice can access. At some point, Alice may want to share the genome data with a peer (Bob) for collaborative work. Thanks to the re-encryption mechanism of IBPRE, Alice can generate a re-encryption key using the identity of Bob and give this key to the cloud. Then the cloud can convert the ciphertexts intended for Alice into ciphertexts for Bob. When getting online, Bob can download the ciphertexts from the cloud and decrypt out the genome data using his private key.

IBPRE provides the ciphertext conversion from a recipient to a new recipient, so as to achieve data sharing among different users. However, there exist some limitations in IBPRE. Specifically, in IBPRE, Alice can share encrypted data with only one recipient at a time. If Alice wants to share data with more recipients, she has to generate a re-encryption key for each recipient respectively, which would incur a considerable computation cost. Besides, IBPRE only provides an “all-or-nothing” data sharing mechanism, which means that Alice shares either all of her data or nothing. These limitations would render IBPRE impractical for some more complicated applications. To further illustrate this point, we consider the following scenario.

Suppose that Alice can access the encrypted genome data uploaded by different volunteers. Before uploading, volunteers have labeled the genome data with descriptive tags, e.g., a volunteer uses a set of tags {“Female”, “Age”=30, “Diabetes”} to indicate that the genome data belongs to a 30-year-old woman who has the inherited diabetes. For collaborative work with a group of peers, Alice would like to share some genome data with the peers. For instance, Alice wants to share the genome data of 20-40 years old women who may have diabetes or heart disease, i.e., the data whose tags meet a policy: “Female” AND “Age” ∈ [20, 40] AND (“Diabetes” OR “Heart Disease”) (see Fig. 2). Since the genome data have already been encrypted, the peers cannot directly access them. Therefore, Alice may need a flexible re-encryption mechanism that can convert the encrypted genome data satisfying the access policy into the ciphertexts that can be decrypted by the group of peers.

IBPRE cannot fit well to this scenario due to its single-recipient and “all-or-nothing” limitations in data sharing. To mitigate the “all-or-nothing” problem, Shao *et al.* proposed the notion of identity-based conditional PRE (IBCPRE [12]), in which Alice can specify a condition for a re-encryption key so that the proxy can only convert the encrypted data matching the condition. Unfortunately, IBCPRE does not allow to specify multiple conditions in a re-encryption key, which implies that Alice still has to generate many re-encryption keys if needing to specify multiple conditions

for data sharing. Moreover, IBCPRE only supports single-recipient data sharing, thus the data cannot be simultaneously accessed by a group of users. Another potential solution may be to use the conditional identity-based broadcast PRE (CIBPRE [9]) which supports data sharing with multiple recipients. Similar to IBCPRE, however, CIBPRE allows only one condition to be specified in a re-encryption key, which also limits the flexibility for data sharing.

1.1 Our Contributions

In this paper, we investigate how to achieve secure and flexible data sharing in cloud computing. For the first time, we propose a novel notion called policy-based broadcast access authorization (PBAA). We also define the notion (including scheme definition and security model) of PBAA and instantiate a concrete PBAA scheme. To the best of our knowledge, the PBAA scheme is the first one that simultaneously provides the following features.

Broadcast Data Sharing: Before outsourcing data to the cloud, the data owner can specify a group of users and encrypt data with the identities of these users. Then the data outsourced to the cloud is stored in a broadcast-ciphertext format such that all the designated users can simultaneously access the data.

Policy-based Re-encryption: If deciding to share data encrypted in the broadcast ciphertexts with a new group of users, the data owner can formulate an access policy to generate a delegation key. Given this key, the cloud can convert any initial ciphertext satisfying the access policy into a new ciphertext such that the underlying data can be accessed by the new group of users.

Data Privacy Protection: The privacy of outsourced data is well protected by the PBAA scheme. Specifically, any user without the correct private key cannot access encrypted data; and even having the delegation key, the cloud cannot access encrypted data, nor deduce a valid delegation key to re-encrypt the ciphertext that is not specified by the data owner.

We also conduct theoretical and experimental analyses on the PBAA scheme and make comparisons with some related schemes. The results show that the PBAA scheme achieves an efficient and flexible access authorization for data sharing, without incurring significant computational costs to data users or cloud servers.

1.2 Paper Organization

The rest of the paper is organized as follows. Sec. 2 introduces the related work and Sec. 3 presents the PBAA system architecture and security requirements. The PBAA framework and the security are formalized in Sec. 4. Sec. 5 describes the PBAA scheme and Sec. 6 proves its security. Sec. 7 analyzes the PBAA scheme both theoretically and experimentally. Sec. 8 concludes the paper.

2 RELATED WORK

Broadcast Encryption. Fiat and Naor first proposed the notion of broadcast encryption [13] where a data owner can encrypt a message to a group of recipients at a time. Traditional broadcast encryption (e.g., [14], [15], [16]) relies on a third party to manage public-key certificates of all users, which would incur a single-point problem. Then Delerablée [17] proposed the identity-based broadcast encryption (IBBE) scheme that avoids the use of public-key certificates by allowing any public identities to serve as public keys. Sakai and Furukawa [18] also presented an IBBE scheme with small-size ciphertexts and private keys. To revoke the access rights of a recipient to a broadcast ciphertext, Lai *et al.* proposed a

revocable IBBE scheme which offers the data owner revocation of ciphertext [19]. Subsequently, Lai *et al.* proposed a fully privacy-preserving revocable IBBE that achieves both recipient revocation and user identity protection [20]. Similar with IBBE, attribute-based encryption (ABE) [21], [22] also allows to encrypt a message to a group of users. Moreover, ABE provides a fine-grained access control on data [23], i.e., a data owner can specify an access policy to encrypt data so that only the users whose attributes satisfy the access policy can access the data.

Identity-based proxy re-encryption. Broadcast encryption achieves a secure multi-recipient data sharing, while it is hard to support sharing encrypted data with new recipients beyond those initially designated. To address this issue, Blaze *et al.* proposed the concept of proxy re-encryption (PRE) [5]. In PRE, a proxy can be authorized to transform Alice's ciphertexts into Bob's ciphertexts, so that the encrypted data can be shared from Alice to Bob. Later, Green and Ateniese [6] extended PRE to identity-based PRE (IBPRE), which mitigates the public-key certificate management problem. To resist collusion attack from the proxy and the users authorized to access re-encrypted data, Zhang *et al.* [24] presented a collusion-resistant IBPRE scheme. Shao and Cao [25] proposed a multi-time IBPRE scheme in which a ciphertext can be re-encrypted for multiple times. This multi-time IBPRE allows an authorized user to further share the re-encrypted ciphertext to others. To achieve encrypted data sharing with a group of recipients, Ge *et al.* [26] proposed an identity-based broadcast PRE scheme which can convert a ciphertext for a set of recipients into a new ciphertext for a new set of recipients. Recently, Deng *et al.* [7] also proposed an IBPRE that can convert a single-recipient ciphertext into a multi-recipient broadcast ciphertext.

Conditional proxy re-encryption. While providing a feasible method to share encrypted data, (identity-based) PRE has a privacy risk that the proxy is given too much power in the re-encryption. The proxy, with the re-encryption key, can convert all the ciphertexts of a data owner; however, the data owner sometimes may only want to share a subset of ciphertexts. To achieve this goal, Weng *et al.* proposed conditional PRE [27], whereby a data owner can specify a condition in the re-encryption key such that only the ciphertexts satisfying the condition can be transformed by the proxy. Vivek *et al.* [28] enhanced the efficiency of Weng *et al.*'s scheme by reducing the number of bilinear pairing operations. Chu *et al.* [29] proposed a conditional broadcast PRE scheme which allows to generate multi-recipient ciphertexts in the conditional PRE settings. By mixing identity-based encryption and conditional PRE, Shao *et al.* first formalized the notion of identity-based conditional PRE (IBCPRE) [12]. Yin *et al.* [8] proposed a broadcast IBCPRE scheme while a ciphertext can be shared only once. Liang *et al.* [30] proposed a multi-time IBCPRE scheme which allows the proxy to further convert re-encrypted ciphertexts. Xu *et al.* [9] proposed a conditional identity-based broadcast PRE scheme that allows to conditionally re-encrypt a broadcast ciphertext for multiple times.

Fine-grained conditional proxy re-encryption. The above conditional (identity-based) PRE schemes enable data owners to choose a subset of ciphertexts to share, but the data sharing mechanism is somewhat limited. Specifically, the data owner can specify only one condition in the re-encryption key and the re-encryption works if the target ciphertext contains the same condition. To improve the flexibility for data sharing, Fang *et al.* [31] proposed a fuzzy conditional PRE scheme, in which a re-encryption key and a ciphertext are respectively associated with a set of conditions,

and the re-encryption works when the two sets are close to each other as measured by a "set overlap" distance metric. Later, Yang *et al.* [32] proposed a ciphertext-policy conditional PRE scheme, where each ciphertext is associated with an access tree and a ciphertext can be re-encrypted if its access tree is satisfied. Yang *et al.*'s scheme improves the flexibility of the scheme in [31], though it would suffer the complicated public-key certificate management. Ge *et al.* [33] presented a fine-grained identity-based conditional PRE scheme, where each ciphertext is labeled with a set of conditions and each re-encryption key is associated with an access tree that specifies which type of ciphertexts the proxy can transform. Ge *et al.*'s scheme achieves a more flexible data sharing as a data owner can specify an access tree rather than a set of conditions in the re-encryption key; unfortunately, Ge *et al.*'s scheme only supports single-recipient data sharing.

Recently, Ge *et al.* [11] proposed a fine-grained identity-based proxy broadcast re-encryption scheme which extends the scheme in [33] to support multi-recipient data sharing. The security of the proposed scheme in [11] has been empirically analyzed, while, unfortunately, the security cannot be formally verified since neither threat model nor mathematical proof is given. Huang *et al.* [10] also proposed an identity-based broadcast PRE supporting fine-grained data sharing. In their proposal, a user associated with a set of conditions is able to generate a re-encryption key for the same (or a subset of the) condition set, which, nevertheless, would limit the flexibility of data sharing. In this paper, we aim at providing a much more flexible data sharing scheme in which a data owner can generate a re-encryption key by formulating an access policy over multiple conditions; this re-encryption key then can be used to transform the ciphertexts satisfying the access policy. Different from the work [33] that also achieves policy-based re-encryption, our proposed scheme allows to convert ciphertexts for multiple times, which enables a data user to further share the re-encrypted data with others. Besides, our proposed scheme can be proved to be secure under a formal security model when compared to [11]. Overall, this paper proposes a provably secure identity-based PRE scheme that simultaneously achieves fine-grained, multi-time, and multi-recipient sharing of encrypted data.

3 ARCHITECTURE AND SECURITY REQUIREMENTS OF PBAA SYSTEM

3.1 Architecture

The PBAA system consists of four entities (as shown in Fig. 3), that is, a central authority (CA), cloud service provider (CSP), data owners and data users. CA is a fully trusted party responsible for publishing system public key and responding to registration requests from data owners and data users. CSP has abundant resources to provide storage and computation services. Specifically, CSP provides storage service for data owners to store the ciphertexts of their data, and provides computing service to transform the ciphertexts into re-encrypted ones. Thus, CSP stores both original ciphertexts and re-encrypted ciphertexts. In real-world scenarios, an organization can buy the storage and computing services provided by CSP, and the IT center of the organization plays the role of CA. Then all the employees registered at CA can make use of the cloud services.

Data owners can outsource their data to CSP for data sharing. Specifically, a data owner can designate a set S of identities of the intended data users and a set L of descriptive conditions (e.g., keywords of the data content), encrypting his data with these

two sets, and then outsources the resulting (original) ciphertext to CSP. When data users designated in the set S are online, they can retrieve the ciphertext from CSP and decrypt it using their private keys. If the data owner or an authorized data user in S wants to share some data to a new group (denoted by S') of users, the data owner formulates an access structure $\mathbb{A}$ over conditions and then generates a delegation key (i.e., re-encryption key) with $\mathbb{A}$ and the identities of the users in S' . With such delegation key, CSP converts the original ciphertexts with conditions satisfying the access structure into the re-encrypted ciphertexts such that all the users in S' can decrypt using their own private keys.

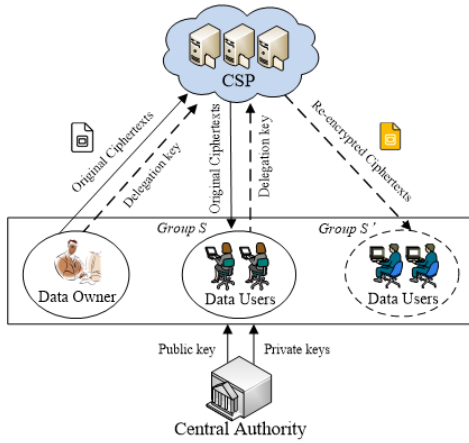


Fig. 3: System Architecture of PBAA

3.2 Threat Model and Security Requirements

A PBAA system confronts several types of active attacks. First, malicious CSP or hackers intruding the cloud storage system would search and access the data outsourced by data owners. Second, unauthorized users would impersonate data owners or authorized users to access the outsourced data; even worse, they may collude with CSP to download and read the data. Third, CSP may abuse the delegation keys to try to re-encrypt the ciphertexts beyond the scope of the delegations. Considering these realistic attacks, we identify the security goals of PBAA as follows.

Privacy of outsourced data. If the data outsourced to CSP has been encrypted, then only the data users having the correct private keys can access. Also, the data that has been re-encrypted can only be accessed by the data users having the correct private keys. In other words, any encrypted data is unreadable to CSP and unauthorized data users (who do not have correct private keys).

Specificity of delegation key. A delegation key issued by a data owner can only be used to convert the specified ciphertexts (i.e., the ciphertexts with conditions satisfying the access structure). Even CSP or authorized data users cannot abuse the delegation key to re-encrypt unspecified ciphertexts, nor can they deduce a valid delegation key to re-encrypt unspecified ciphertexts.

4 DEFINITIONS

4.1 Framework of PBAA system

Based on the system architecture described in Sec. 3.1, we now define the PBAA system. A PBAA system involves seven algorithms, namely, Setup, Register, Encrypt, DKGen, ReEnc, Dec1 and Dec2. At the beginning, CA initializes the system by calling the

Setup algorithm to generate system public key PK and master secret key MSK . It publishes PK to other parties while keeps MSK secret. Whenever a user requests to join in the system, the user should ask CA for registration. For a user allowed to join in, CA first determines a unique identity ID and then generates a privacy key with ID by calling the Register algorithm.

Before outsourcing data to the cloud, the data owner can apply the Encrypt algorithm to protect data privacy. Specifically, the data owner first specifies the set S of recipients' identities and the set L of descriptive conditions, and then encrypts the data with S and L . Then the data owner outsources the resulting ciphertext CT to CSP. When the data owner wants to share the encrypted data to a new group of recipients, the data owner only needs to determine the set S' of the new recipients' identities and formulate an access policy $\mathbb{A}$ that specifies which type of data he wishes to share. Then the data owner generates a delegation key with S' and $\mathbb{A}$ by calling the DKGen algorithm. With the delegation key, CSP converts any ciphertext with set L satisfying $\mathbb{A}$ into a re-encrypted ciphertext CT' by calling the ReEnc algorithm. When a data user gets online, he can download a ciphertext from CSP and try to decrypt it. For an original ciphertext CT (resp. a re-encrypted ciphertext CT'), the data user calls the Dec1 (resp. Dec2) algorithm to recover the underlying data if his identity is specified in the set S (resp. S').

Specifically, the Setup, Register, Encrypt, DKGen, ReEnc, Dec1 and Dec2 algorithms of the PBAA system are described as follows.

- $(PK, MSK) \leftarrow \text{Setup}(1^\ell, N)$: Given a security parameter ℓ and the maximum number N of recipients for a ciphertext, CA runs this setup algorithm to generate system public key PK and master secret key MSK . CA publishes PK while keeps MSK secret.
- $SK_{ID} \leftarrow \text{Register}(PK, MSK, ID)$: Given PK , MSK and an identity ID , CA runs this registration algorithm to output a private key SK_{ID} for user ID .
- $CT \leftarrow \text{Encrypt}(PK, M, S, L)$: Given PK , a message M , a set S of identities (where $|S| \leq N$) and a set L of conditions, the data owner runs this encryption algorithm to output an original ciphertext CT .
- $DK_{ID \rightarrow S'|\mathbb{A}} \leftarrow \text{DKGen}(PK, SK_{ID}, S', \mathbb{A})$: Given PK , SK_{ID} , a set S' of identities (where $|S'| \leq N$) and an access policy $\mathbb{A}$, the user ID runs this delegation algorithm to output a delegation key $DK_{ID \rightarrow S'|\mathbb{A}}$.
- $CT'/\perp \leftarrow \text{ReEnc}(PK, CT, S, L, DK_{ID \rightarrow S'|\mathbb{A}})$: Given PK , an original ciphertext CT generated under S and L , and a delegation key $DK_{ID \rightarrow S'|\mathbb{A}}$, CSP runs this re-encryption algorithm to output a re-encrypted ciphertext CT' for S' if $ID \in S$ and L satisfies $\mathbb{A}$, or a false symbol $\perp$ otherwise.
- $M/\perp \leftarrow \text{Dec1}(PK, SK_{ID}, CT, S)$: Given PK , SK_{ID} , and an original ciphertext CT of S , the data user ID runs this decryption algorithm to output the message M if $ID \in S$, or a false symbol $\perp$ otherwise.
- $M/\perp \leftarrow \text{Dec2}(PK, SK_{ID'}, CT', S')$: Given PK , $SK_{ID'}$, and a re-encrypted ciphertext CT' of S' , the data user ID' runs this decryption algorithm to output the message M if $ID' \in S'$, or a false symbol $\perp$ otherwise.

Correctness : The correctness of the PBAA system says that if each party follows the scheme, then any correctly generated ciphertext (including original ciphertext and re-encrypted cipher-

text) can be decrypted by a valid private key. Formally, for any $(PK, MSK) \leftarrow \text{Setup}(1^\ell, N)$, the following two conditions are satisfied.

- For any original ciphertext $CT \leftarrow \text{Encrypt}(PK, M, S, L)$ and any private key $SK_{ID} \leftarrow \text{Register}(PK, MSK, ID)$, if $ID \in S$, then $\text{Dec1}(PK, SK_{ID}, CT, S)$ always outputs the plaintext M .
- For any re-encrypted ciphertext $CT' \leftarrow \text{ReEnc}(PK, S, L, CT, DK_{ID \rightarrow S'|\mathbb{A}})$ and any private key $SK_{ID'} \leftarrow \text{Register}(PK, MSK, ID')$, where $CT \leftarrow \text{Encrypt}(PK, M, S, L)$, $DK_{ID \rightarrow S'|\mathbb{A}} \leftarrow \text{DKGen}(PK, SK_{ID}, S', \mathbb{A})$, and $SK_{ID} \leftarrow \text{Register}(PK, MSK, ID)$, if $ID \in S$, L satisfies $\mathbb{A}$ and $ID' \in S'$, then $\text{Dec2}(PK, SK_{ID'}, CT', S')$ always outputs the plaintext M .

The first correctness means that any original ciphertext can be correctly decrypted by the intended users. The second correctness defines what is a properly re-encrypted ciphertext and who can correctly decrypt it. Specifically, the second correctness describes that only when $ID \in S$ and L satisfies $\mathbb{A}$, the original ciphertext generated under S and L can be properly re-encrypted by the delegation key $DK_{ID \rightarrow S'|\mathbb{A}}$. Further, the second correctness defines that only the data users with identities specified in S' can correctly decrypt the re-encrypted ciphertext.

4.2 Formal Security Definition

In this section, we give a formal security definition for the PBAA system. We assume that the cloud is a semi-honest party, which means that it will honestly execute the assigned tasks but cannot guarantee the confidentiality of user data. We also assume that the algorithms at non-cloud side (e.g., key generation, delegation, encryption and decryption algorithms) will be correctly executed.

To capture the realistic attacks described in Sec. 3.2, we define the indistinguishability of ciphertexts against selective identity/condition-set and chosen plaintext attack (IND-sSet-CPA) for the PBAA system. In the IND-sSet-CPA security, a probabilistic polynomial-time (PPT) adversary is defined to model the malicious CSP and unauthorized users. The adversary is able to acquire users' private keys and CSP's delegation keys. It can also obtain the system public key after system initialization. We note that this definition of the adversary has already captured the collusion attack from the malicious CSP and unauthorized users. For such an adversary, the IND-sSet-CPA security guarantees that it cannot distinguish which one of two messages is encrypted in an original ciphertext, if the adversary does not have the private keys of the receivers of both the original ciphertext and its re-encrypted ciphertexts. In other words, an original ciphertext or a re-encrypted ciphertext would not reveal any useful information about the underlying message to the adversary.

The IND-sSet-CPA security for the PBAA system is defined by the following security game played between the adversary and the challenger.

Init: The adversary $\mathcal{A}$ chooses a set S^* of identities (where $|S^*| \leq N$) and a set L^* of conditions that it would like to attack. $\mathcal{A}$ sends S^* and L^* to the challenger.

Setup: The challenger runs the Setup algorithm to obtain the system public key PK and master secret key MSK . It gives PK to the adversary $\mathcal{A}$.

Query 1: $\mathcal{A}$ adaptively makes the following queries:

- *Registration*(ID_i). $\mathcal{A}$ can ask a private key for any identity ID_i with the restriction $ID_i \notin S^*$. In response, the challenger runs the algorithm $\text{Register}(PK, MSK, ID_i)$ to generate private key SK_{ID_i} and returns it to $\mathcal{A}$.
- *Delegation*($ID_i \rightarrow S'|\mathbb{A}$). $\mathcal{A}$ can ask a delegation key for an identity ID_i , an identity set S' and an access policy $\mathbb{A}$, with the restriction that $\mathcal{A}$ can make at most one of the queries: *Registration*(ID') for any $ID' \in S'$ and *Delegation*($ID_i \rightarrow S'|\mathbb{A}$) for any $\mathbb{A}$ such that L^* satisfies $\mathbb{A}$ when $ID_i \in S^*$. In response, the challenger runs the algorithm $\text{DKGen}(PK, SK_{ID_i}, S', \mathbb{A})$ to generate a delegation key $DK_{ID_i \rightarrow S'|\mathbb{A}}$ and returns it to $\mathcal{A}$.

Challenge: $\mathcal{A}$ outputs two equal-length messages M_0 and M_1 to the challenger. The challenger flips a coin $b \in \{0, 1\}$, encrypts M_b under S^* and L^* via the Encrypt algorithm, and then returns the resulting ciphertext CT^* to $\mathcal{A}$.

Query 2: $\mathcal{A}$ adaptively makes queries as in Query 1.

Guess: The adversary $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.

The advantage of the adversary $\mathcal{A}$ in this game is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{IND-sSet-CPA}} = |\Pr[b = b'] - 1/2|.$$

Definition 1. A PBAA scheme is IND-sSet-CPA secure if any PPT $\mathcal{A}$ which plays the above game with the challenger has only a negligible advantage in winning the game.

The definitions of PBAA and its security show how the functionalities of “multi-recipient data sharing” and “fine-grained access authorization” are defined in the concept of PBAA. The functionality “multi-recipient data sharing” is straightforward: a ciphertext of PBAA can be decrypted by a group of intended users. The functionality “fine-grained access authorization” definition is somewhat sophisticated. We explain it in more details. The second correctness of PBAA defines that “ L satisfies $\mathbb{A}$ ” is one of sufficient conditions for CSP to correctly re-encrypt the original ciphertext. Moreover, the IND-sSet-CPA security defines that when L does not satisfy $\mathbb{A}$, the delegation key is unable to re-encrypt the ciphertext. Therefore, in the PBAA system, the data owner can formulate an access structure to specify which data he wishes to share with intended users, which achieves a fine-grained access authorization for encrypted data.

We note that the security defined by Definition 1 does not require to protect the confidentiality of delegation keys. That is to say, the delegation keys can be known to adversarial attackers, since in the security game defined above, the adversary is allowed to make queries for delegation keys of its choice. This captures the realistic attack that some hackers may defraud or intrude the cloud to obtain any delegation keys, and also captures the attack that the cloud may intentionally leak the delegation keys for some economic benefits. The IND-sSet-CPA secure PBAA system guarantees that even the delegation keys are known to attackers, the privacy of data stored in the cloud can still be protected, as long as the corresponding private keys are not exposed.

5 PROPOSED PBAA SCHEME

5.1 Basic Idea

Unlike the previous approach (commonly used in [12], [9], [32], [33]) that specifies a single condition in the delegation key, we propose a flexible access authorization mechanism by introducing an access policy over multiple conditions into the delegation key. This mechanism is fulfilled by seamlessly integrating the IBBE scheme

[17] and the KP-ABE scheme [22]. Specifically, a (an original) ciphertext of PBAA involves two parts: an IBBE ciphertext and a KP-ABE ciphertext. The IBBE ciphertext is an encryption of the plaintext while the KP-ABE ciphertext is generated with different conditions. To resist collusion attack, these two parts are bounded by the same random value. We also follow the delegation-key generation technique widely used in many PRE schemes [6], [34], [9], [26], [35]. With this technique, the data owner generates a delegation key by first encrypting a secret value via the IBBE encryption and then blinding his private key with the secret value under an access policy; in the decryption, data users first obtain the secret value and then apply it to recover the plaintext.

However, there is a major challenge in applying the above techniques to construct the PBAA scheme. That is, the data owner is hard to realize the access policy in the delegation key without knowing the system master secret key. Specifically, to generate a functional delegation key, the data owner should split the master secret key into shares and assign each share to a condition involved in the access policy. It is unable for the data owner to do this without having the master secret key. To overcome this challenge, we present a novel approach for applying the linear secret sharing scheme (LSSS) to split the data owner's private key according to the access policy. Since the master secret key is properly embedded in the private key, the shares of the secret can still be obtained. These shares then can be used in the re-encryption to convert the ciphertexts that satisfy the access policy.

5.2 Background

The PBAA scheme is built on bilinear groups. Suppose $\mathbb{G}$ and $\mathbb{G}_T$ are two finite cyclic groups of prime order p . Let g be a generator of $\mathbb{G}$. The group $\mathbb{G}$ is a bilinear group if there exists an efficient bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ such that: i) for all $u, v \in \mathbb{G}$ and $a, b \in \mathbb{Z}_p$, $e(u^a, v^b) = e(u, v)^{ab}$ and ii) $e(g, g) \neq 1$.

DBDH assumption [36]. Our PBAA scheme will rely on DBDH assumption. Suppose that $\mathbb{G}$ is a bilinear group with prime order p . The DBDH assumption says that no PPT adversary can distinguish two tuples $(g^a, g^b, g^c, e(g, g)^{abc})$ and $(g^a, g^b, g^c, e(g, g)^r)$ where $a, b, c, r \in \mathbb{Z}_p$ are randomly selected.

Access structure [37]. In the PBAA scheme, the access policy associated with a delegation key will be represented as an access structure. Let $\{P_1, P_2, \dots, P_n\}$ be a set of parties. A collection $\mathbb{A} \subseteq 2^{\{P_1, P_2, \dots, P_n\}}$ is monotone if for $\forall B, C$, we have that $C \in \mathbb{A}$ holds if $B \in \mathbb{A}$ and $B \subseteq C$. An access structure (respectively, monotone access structure) is a collection (respectively, monotone collection) $\mathbb{A}$ of non-empty subsets of $\{P_1, P_2, \dots, P_n\}$, i.e., $\mathbb{A} \subseteq 2^{\{P_1, P_2, \dots, P_n\}} \setminus \{\emptyset\}$. The sets in $\mathbb{A}$ are called the authorized sets, and the sets not in $\mathbb{A}$ are called the unauthorized sets.

In our scheme, the role of parties mentioned above is played by the conditions (e.g., keywords). Then an access structure in our scheme is a collection of sets of conditions. If a set L of conditions is in an access structure $\mathbb{A}$ (i.e., $L \in \mathbb{A}$), we call it authorized set.

LSSS [37]. To realize the access structure, we employ the LSSS in the delegation-key generation. A secret-sharing scheme Π over a set of parties $\mathcal{P}$ is called linear (over $\mathbb{Z}_p$) if: i) the shares for each party form a vector over $\mathbb{Z}_p$; ii) there exists a matrix A called the share-generating matrix for Π , where A has l rows and q columns. For all $i = 1, \dots, l$, the i -th row (denoted by A_i) of A is labeled by a party $\rho(i)$, where ρ is a function from $\{1, \dots, l\}$ to $\mathcal{P}$. When we consider the column vector $\vec{v} = (\delta, y_2, \dots, y_q)$, where $\delta \in \mathbb{Z}_p$ is the secret to be shared, and $y_2, \dots, y_n \in \mathbb{Z}_p$ are randomly chosen, then $\lambda_i = A_i \cdot \vec{v}$ is the share for party $\rho(i)$.

According to [37], each LSSS enjoys the reconstruction feature. That is, given an LSSS (A, ρ) for access structure $\mathbb{A}$ and an authorized set $L \in \mathbb{A}$, there must exist constants $\{\omega_i \in \mathbb{Z}_p\}$ that can help to recover the secret δ .

5.3 Construction

The PBAA scheme is built from the IBBE scheme [17] and consists of the following algorithms.

5.3.1 System Setup

In this stage, CA initializes the PBAA scheme by calling the Setup algorithm. Suppose that a company buys cloud services for its employees to store and share business data. Then the IT center of the company can play the role of CA to initialize the PBAA scheme so that all the employees can apply the PBAA scheme to protect the privacy of the business data. By running Setup algorithm, CA generates system public key PK and master secret key MSK as follows.

$(PK, MSK) \leftarrow \text{Setup}(1^\ell)$: Let ℓ be the security parameter and N be the maximum number of receivers for one encryption. CA first generates the bilinear map parameters $(p, g, \mathbb{G}, \mathbb{G}_T, e)$ with ℓ . Then it randomly selects $\alpha \in \mathbb{Z}_p$, $\mu, w, u, h, v \in \mathbb{G}$, and computes $\mu_1 = \mu^\alpha$ and $g_1 = g^\alpha, g_2 = g^{\alpha^2}, \dots, g_N = g^{\alpha^N}, w_1 = w^\alpha, \dots, w_N = w^{\alpha^N}, v_1 = v^\alpha, \dots, v_N = v^{\alpha^N}$. In addition, CA chooses two hash functions $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ and $F : \mathbb{G}_T \rightarrow \mathbb{G}$. Then, CA defines the system public key and master secret key as $PK = (p, g, g_1, \dots, g_N, w, w_1, \dots, w_N, v, v_1, \dots, v_N, \mu_1, u, h, e(g, \mu), H, F)$ and $MSK = (\mu, \alpha)$. At last, CA publishes PK to other parties while keeps MSK secret.

5.3.2 Registration

In the registration stage, CA first checks whether a user is allowed to join in the system. For example, the IT center (CA) of a company checks whether a user requesting to use the cloud services is a valid employee. If yes, CA generates a private key and sends it to the user as an authorized credential to access the data stored in the cloud. To generate a private key, CA first determines a unique identity ID for the user and then runs the following Register algorithm.

$SK_{ID} \leftarrow \text{Register}(PK, MSK, ID)$: Given the system public key PK , master secret key MSK and the identity ID , CA computes the private key for ID as $SK_{ID} = \mu^{\frac{1}{\alpha + H(ID)}}$.

Then CA gives SK_{ID} to the user ID through a secure channel (e.g., SSL).

5.3.3 Encryption

With the PBAA scheme, a data owner can securely share data with a group of recipients. Specifically, the data owner first determines a set S of identities of the recipients. We assume that $S = \{ID_i\}_{1 \leq i \leq N}$ in which ID_i denotes the identity of the i -th recipient. Meanwhile, the data owner specifies a set $L = \{L_1, L_2, \dots, L_m\}$ of conditions to describe the data, e.g., an employee can use a set $L = \{\text{"Marketing department"}, \text{"Electronic Product"}, \text{"Turnover"}, \text{"Feb/2021"}\}$ to indicate the electronic-product sales turnover data of the marketing department in Feb 2021. To achieve efficient data encryption, the key encapsulation technique [38], [7] can be also adopted. With such technique, a data owner first encrypts the data with a symmetric encryption scheme (e.g., AES) and then encrypts the symmetric-encryption

key with an asymmetric encryption. Thus the cost of the asymmetric encryption will be independent of the data size. In the application of the PBAA scheme, a data owner first picks a random key $M \in \mathbb{G}_T$ and then encrypts M with the sets S and L by calling the following Encrypt algorithm.

$CT \leftarrow \text{Encrypt}(PK, M, S, L)$: Given the system public key PK , the set S of identities, the set L of conditions, and the message $M \in \mathbb{G}_T$ to be encrypted, the data owner first chooses a random $s \in \mathbb{Z}_p$ and then computes

$$C_0 = Me(g, \mu)^s, \quad C_1 = g^s \prod_{ID_i \in S} (\alpha + H(ID_i)),$$

$$C_2 = \mu_1^{-s}, \quad C_3 = v^s \prod_{ID_i \in S} \frac{\alpha + H(ID_i)}{H(ID_i)}.$$

In addition, for each condition L_j in L , the data owner chooses a random value $r_j \in \mathbb{Z}_p$ and computes

$$C_{4,j} = g^{r_j}, \quad C_{5,j} = (u^{H(L_j)} h)^{r_j} \cdot w^{-s \prod_{ID_i \in S} (\alpha + H(ID_i))}.$$

The data owner then defines the ciphertext as $CT = (C_0, C_1, C_2, C_3, \{C_{4,j}, C_{5,j}\}_{j=1}^m)$ and sends it to CSP.

5.3.4 Delegation

After a volume of data have been encrypted and outsourced to the cloud, the data owner (or authorized user in S) can still share some data with a new group of recipients. Suppose that the data owner wants to share the data regarding the turnover and development cost of an electronic product in the first quarter. The data owner first formulates an access policy $\mathbb{A}$ = “Electronic product” AND “Jan/01/2021-Mar/31/2021” AND (“Turnover” OR “Development cost”), and determines the set S' of the new recipients' identities. Then the data owner generates a delegation key with $\mathbb{A}$ and S' by calling the following DKGen algorithm.

$DK_{ID \rightarrow S'} \leftarrow \text{DKGen}(PK, SK_{ID}, S', \mathbb{A})$: Suppose that $S' = (ID'_1, ID'_2, \dots, ID'_{n'})$ is the set of n' identities and the access policy $\mathbb{A}$ involves l conditions. The data owner first chooses random values $\delta, s' \in \mathbb{Z}_p$ and computes

$$d_0 = g^\delta \cdot F(e(g, \mu)^{s'}), \quad d_1 = g^{s' \prod_{ID'_i \in S'} (\alpha + H(ID'_i))}, \quad d_2 = \mu_1^{-s'}.$$

Then the data owner generates an LSSS (A, ρ) for $\mathbb{A}$, where A is an $l \times q$ matrix and the function ρ maps each row of A to a condition. Choose random values $y_2, y_3, \dots, y_q \in \mathbb{Z}_p$ to form a vector $\vec{\delta} = (\delta, y_2, \dots, y_q)$. For the k -th row $A_k = (A_{k,1}, \dots, A_{k,q})$ of the matrix A , compute a share of δ as $\lambda_k = A_k \cdot \vec{\delta}$.

In addition, the data owner picks random values $y'_2, y'_3, \dots, y'_q \in \mathbb{Z}_p$ and computes

$$(SK_{ID})^{A_{k,1}} g^{y'_2 A_{k,2} + \dots + y'_q A_{k,q}}$$

for each $k \in [l]$. If we can implicitly define $SK_{ID} = g^x$ for an unknown value $x \in \mathbb{Z}_p$, the above product can be written as $(g^x)^{A_{k,1}} \cdot g^{y'_2 A_{k,2} + \dots + y'_q A_{k,q}} = g^{A_k \cdot \vec{x}}$, where $\vec{x} = (x, y'_2, \dots, y'_q)$. For simplicity, we set $\psi_k = A_k \cdot \vec{x}$.

The data owner proceeds to choose a random value $t_k \in \mathbb{Z}_p$ for each k from 1 to l , and computes

$$d_{3,k} = g^{\psi_k} \cdot v^{\frac{\lambda_k}{H(ID)}} \cdot w^{t_k}, \quad d_{4,k} = (u^{\rho(k)} h)^{-t_k}, \quad d_{5,k} = g^{t_k}.$$

Then the data owner defines the delegation key as $DK_{ID \rightarrow S'} = (d_0, d_1, d_2, \{d_{3,k}, d_{4,k}, d_{5,k}\}_{k=1}^l)$ and sends it to CSP.

5.3.5 Re-Encryption

Upon receiving the delegation key $DK_{ID \rightarrow S'}$ from the data owner, CSP applies it to transform the original ciphertexts to be accessible to a new set S' of recipients. CSP first searches the original ciphertexts generated under S and L such that $ID \in S$ and $L \in \mathbb{A}$. Here, the condition $ID \in S$ implies that the delegation key can only be used to re-encrypt the data owner's ciphertexts, and the condition $L \in \mathbb{A}$ means that only a subset of ciphertexts, specified by the data owner, can be re-encrypted. After finding all the ciphertexts satisfying the two conditions, CSP uses $DK_{ID \rightarrow S'}$ to re-encrypt each ciphertext by calling the following ReEnc algorithm.

$CT' / \perp \leftarrow \text{ReEnc}(PK, CT, S, L, DK_{ID \rightarrow S'})$: Given the system public key PK , the delegation key $DK_{ID \rightarrow S'}$ and an original ciphertext $CT = (C_0, C_1, C_2, C_3, \{C_{4,j}, C_{5,j}\}_{j=1}^m)$ that is generated under the sets S and L , if L does not satisfy $\mathbb{A}$, CSP outputs a false symbol $\perp$. Otherwise, due to the linear reconstruction property of LSSS, CSP can find constants $\{\omega_k \in \mathbb{Z}_p\}$ such that $\sum_{k \in K} \omega_k \lambda_k = \delta$ and $\sum_{k \in K} \omega_k \psi_k = x$, where $K = \{k : \rho(k) = H(L_j) \text{ for } L_j \in L\} \subseteq \{1, 2, \dots, l\}$. CSP defines $\tilde{S} = S \setminus \{ID\}$ that is identical to S except without ID . For each $k \in K$, CSP computes

$$B_k = e(d_{3,k}, C_1) \cdot e(d_{4,k}, C_{4,j}) \cdot e(d_{5,k}, C_{5,j}).$$

Then CSP calculates

$$B = \left[\prod_{k \in K} B_k^{\omega_k} \cdot e(g^{\Delta_{\tilde{S}}(\alpha)}, C_2) \right]^{\prod_{ID_i \in \tilde{S}} \frac{1}{H(ID_i)}}$$

where

$$\Delta_{\tilde{S}}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in \tilde{S}} H(ID_i) \right).$$

CSP also sets $C'_0 = d_0, C'_1 = d_1, C'_2 = d_2, C'_3 = C_0/B$, and $C'_4 = C_3$. Finally, CSP defines the re-encrypted ciphertext as $CT' = (C'_0, C'_1, C'_2, C'_3, C'_4)$ and stores it locally.

5.3.6 Decryption

When a data user gets online, he can download a ciphertext from CSP and try to decrypt it using his private key. We note that there are two kinds of ciphertexts stored in the cloud, i.e., original ciphertexts and re-encrypted ciphertexts. Therefore, the decryption procedure can be classified into two cases.

$M / \perp \leftarrow \text{Dec1}(PK, CT, S, SK_{ID})$: This is the decryption algorithm for an original ciphertext. Given an original ciphertext $CT = (C_0, C_1, C_2, C_3, \{C_{4,j}, C_{5,j}\})$ associated with S , if $ID \in S$, the data user holding the private key SK_{ID} can first define a set $\tilde{S} = S \setminus \{ID\}$ and then compute

$$D = \left(e(SK_{ID}, C_1) \cdot e(g^{\Delta_{\tilde{S}}(\alpha)}, C_2) \right)^{\prod_{ID_i \in \tilde{S}} \frac{1}{H(ID_i)}}$$

where

$$\Delta_{\tilde{S}}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in \tilde{S}} H(ID_i) \right)$$

Finally, the data user obtains the plaintext $M = C_0/D$.

$M / \perp \leftarrow \text{Dec2}(PK, CT', S', SK_{ID'})$: This is the decryption algorithm for a re-encrypted ciphertext. Given a re-encrypted ciphertext $CT' = (C'_0, C'_1, C'_2, C'_3, C'_4)$ associated with S' , if

$ID' \in S'$, the data user holding the private key $SK_{ID'}$ can first define a set $\tilde{S}' = S' \setminus \{ID'\}$ and then compute

$$D' = \left(e(SK_{ID'}, C'_1) \cdot e(g^{\Delta_{\tilde{S}'}}(\alpha), C'_2) \right)^{\frac{1}{\prod_{ID'_i \in \tilde{S}'} H(ID'_i)}}$$

where

$$\Delta_{\tilde{S}'}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID'_i \in \tilde{S}'} (\alpha + H(ID'_i)) - \prod_{ID'_i \in \tilde{S}'} H(ID'_i) \right).$$

The data user then computes $g^\delta = C'_0 / F(D')$ and obtains the plaintext $M = C'_3 \cdot e(g^\delta, C'_4)$.

5.3.7 Discussions

In the delegation phase, the only information that the data owner should know about data users are their identities. These identities could be hashes of some secret information (e.g., passwords) of data users. Given the hashes, it is mathematically difficult to compute the secret information reversely; thus, as long as the secret information is kept safely by the data users, the data can be securely shared. To prevent potential identity fraud attacks, moreover, some identity authentication mechanisms could be introduced into the PBAA system. The core of the identity authentication is to let the data user prove the authenticity of his identity to the data owner, before the data owner uses the data user's identity to generate the delegation key. We note that there are already many mature or advanced authentication approaches. Traditionally, password based authentication and smart-card based authentication [39] could be utilized to reduce identity impersonation in the system. Moreover, SSL/TLS based authentication [40] or identity-based authentication protocol [41], [42] can also be adopted. The main idea is to let the data user sign a request with his private key and the data owner verifies whether the signature is valid by using the data user's public key; and if yes, the data owner believes that the request is sent by the real data user. These authentication techniques are compatible with the PBAA scheme and could be integrated with the scheme in specific applications, at the cost of incurring some communication or computation overheads.

In real-world applications, one may also be concerned about the integrity of the delegation keys. To protect the delegation keys from being altered or tampered with, some integrity protection measures such as hash functions and digital signature schemes could be utilized. Specifically, before sending the delegation key to the cloud, the data owner can first use some hash function (e.g., SHA-1) to map the key to a fixed-length hash value, and then sign the value with a digital signature scheme (e.g., BLS short signature [43]). Then the data owner sends the delegation key together with the hash value and the signature to the cloud. The signature enables the cloud to verify whether the delegation key was sent by the real data owner, and the hash value helps to test whether the delegation key has been altered during its transmission. If finding that the received hash value is not equal to the hashing result of the received delegation key, the cloud determines that the transmitted information has been altered and then can ask the data owner to send the delegation key again.

We note that the above identity authentication, hash function and digital signature techniques have been extensively studied (e.g., [44], [45], [43], [46], [47], [39], [40], [42], [41]) and could be used as independent components to equip the PBAA scheme. These techniques, however, are orthogonal to the research domain of this article and we would like to leave the integration of them with the PBAA scheme as our future work.

6 CORRECTNESS AND SECURITY

In this section, we show the correctness of the PBAA scheme and then prove the security of the scheme.

Theorem 1 (Correctness). In a successful registration, a data user always accepts the private key given by CA. CSP always accepts the delegation key if the data owner is honest. If the target (original or re-encrypted) ciphertext has not been altered, then the data users holding the correct private keys always decrypt the ciphertext successfully.

Proof: Since there exist two kinds of ciphertexts (namely, original ciphertexts and re-encrypted ciphertexts), we prove the correctness for these two ciphertexts respectively.

1) For a valid original ciphertext CT and a valid private key SK_{ID} , we prove that $\text{Dec1}(PK, CT, S, SK_{ID})$ always outputs M if $ID \in S$. Without loss of generality, let $CT = (C_0, C_1, C_2, C_3, \{C_{4,j}, C_{5,j}\})$. When $ID \in S$, the Dec1 algorithm run by the data user defines $\tilde{S} = S \setminus \{ID\}$ and computes

$$\begin{aligned} D &= \left(e(SK_{ID}, C_1) \cdot e(g^{\Delta_{\tilde{S}}}(\alpha), C_2) \right)^{\frac{1}{\prod_{ID_i \in \tilde{S}} H(ID_i)}} \\ &= \left(e(\mu^{\frac{1}{\alpha + H(ID)}}, g^{s \prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i))}) \right. \\ &\quad \cdot e(g^{\Delta_{\tilde{S}}}(\alpha), \mu^{-\alpha s}) \left. \right)^{\frac{1}{\prod_{ID_i \in \tilde{S}} H(ID_i)}} \\ &= \left(e(g, \mu)^{s \prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i))} \right. \\ &\quad \cdot e(g, \mu)^{-s \left(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in \tilde{S}} H(ID_i) \right)} \left. \right)^{\frac{1}{\prod_{ID_i \in \tilde{S}} H(ID_i)}} \\ &= e(g, \mu)^s \end{aligned}$$

with

$$\Delta_{\tilde{S}}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in \tilde{S}} H(ID_i) \right).$$

It follows that $M = C_0 / D = M e(g, \mu)^s / e(g, \mu)^s$. We note that this is the standard IBBE decryption as defined in [17].

2) For a valid re-encrypted ciphertext CT' associated with S' and a private key $SK_{ID'}$ associated with ID' , we prove that $\text{Dec2}(PK, CT', S', SK_{ID'})$ always outputs M if $ID' \in S'$. We note that the re-encrypted ciphertext CT' is generated by the re-encryption algorithm. Thus, we first show the correctness of this algorithm. Without loss of generality, let $CT = (C_0, C_1, C_2, C_3, \{C_{4,j}, C_{5,j}\}_{j \in L_j \in L})$ be the original ciphertext and $DK_{ID \rightarrow S'|\mathbb{A}} = (d_0, d_1, d_2, \{d_{3,k}, d_{4,k}, d_{5,k}\}_{k=1}^l)$ be the delegation key. If $ID \in S$ and L satisfies $\mathbb{A}$, the re-encryption algorithm run by CSP defines two sets $\tilde{S} = S \setminus \{ID\}$, $K = \{k : \rho(k) = H(L_j) \text{ for } L_j \in L\} \subseteq \{1, 2, \dots, l\}$ and then computes

$$\begin{aligned} B_k &= e(d_{3,k}, C_1) \cdot e(d_{4,k}, C_{4,j}) \cdot e(d_{5,k}, C_{5,j}) \\ &= e\left(g^{\psi_k} \cdot v^{\frac{\lambda_k}{H(ID)}} \cdot w^{t_k}, g^{s \prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i))}\right) \\ &\quad e\left((u^{\rho(k)} h)^{-t_k}, g^{r_j}\right) \\ &\quad e\left(g^{t_k}, (u^{H(L_j)} h)^{r_j} \cdot w^{-s \prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i))}\right) \\ &= e\left(g^{\psi_k} \cdot v^{\frac{\lambda_k}{H(ID)}}, g^{s \prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i))}\right). \end{aligned}$$

and

$$\begin{aligned} \prod_{k \in K} B_k^{\omega_k} &= \prod_{k \in K} e\left(g^{\psi_k} \cdot v^{\frac{\lambda_k}{H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right)^{\omega_k} \\ &= e\left(g^{\sum_{k \in K} \omega_k \psi_k}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \\ &\quad e\left(v^{\frac{\sum_{k \in K} \omega_k \lambda_k}{H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \\ &= e\left(\mu^{\frac{1}{\alpha + H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \\ &\quad e\left(v^{\frac{\delta}{H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \end{aligned}$$

with $g^{\sum_{k \in K} \omega_k \psi_k} = g^x = \mu^{1/(\alpha + H(ID))}$ and $\sum_{k \in K} \omega_k \lambda_k = \delta$. Then CSP calculates

$$\begin{aligned} B &= \left[\prod_{k \in K} B_k^{\omega_k} \cdot e\left(g^{\Delta_{\tilde{S}}(\alpha)}, \mu^{-\alpha s}\right) \right]^{\frac{1}{\prod_{ID_i \in \tilde{S}} H(ID_i)}} \\ &= \left[e\left(\mu^{\frac{1}{\alpha + H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \cdot e\left(v^{\frac{\delta}{H(ID)}}, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}\right) \right. \\ &\quad \left. e(g, \mu)^{-s(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in S} H(ID_i))} \right]^{\frac{1}{\prod_{ID_i \in \tilde{S}} H(ID_i)}} \\ &= e(g, \mu)^s \cdot e(v^\delta, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}). \end{aligned}$$

with

$$\Delta_{\tilde{S}}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID_i \in \tilde{S}} (\alpha + H(ID_i)) - \prod_{ID_i \in S} H(ID_i) \right).$$

With the above component B , CSP can compute

$$C'_3 = C_0/B = Me(g, \mu)^s/B = M/e(v^\delta, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}).$$

Finally, CSP obtains the re-encrypted ciphertext $CT' = (C'_0, C'_1, C'_2, C'_3, C'_4)$, where $C'_0 = d_0, C'_1 = d_1, C'_2 = d_2$ and $C'_4 = C_3$.

We next show the correctness of the decryption algorithm for CT' . Given a re-encrypted CT' associated with S' and a private key $SK_{ID'}$, if $ID' \in S'$, the Dec2 algorithm run by the data user defines a set $\tilde{S}' = S' \setminus \{ID'\}$ and then computes

$$\begin{aligned} D' &= \left(e(SK_{ID'}, C'_1) \cdot e(g^{\Delta_{\tilde{S}'}(\alpha)}, C'_2) \right)^{\frac{1}{\prod_{ID'_i \in \tilde{S}'} H(ID'_i)}} \\ &= \left(e\left(\mu^{\frac{1}{\alpha + H(ID')}}\right), g^{s' \cdot \prod_{ID'_i \in \tilde{S}'} (\alpha + H(ID'_i))} \right) \\ &\quad e\left(g^{\Delta_{\tilde{S}'}(\alpha)}, \mu^{-\alpha s'}\right)^{\frac{1}{\prod_{ID'_i \in \tilde{S}'} H(ID'_i)}} \\ &= \left(e(g, \mu)^{s' \cdot \prod_{ID'_i \in \tilde{S}'} (\alpha + H(ID'_i))} \cdot e(g, \mu)^{-s'(\prod_{ID'_i \in \tilde{S}'} (\alpha + H(ID'_i)) - \prod_{ID'_i \in S'} H(ID'_i))} \right)^{\frac{1}{\prod_{ID'_i \in \tilde{S}'} H(ID'_i)}} \\ &= e(g, \mu)^{s'} \end{aligned}$$

with

$$\Delta_{\tilde{S}'}(\alpha) = \alpha^{-1} \cdot \left(\prod_{ID'_i \in \tilde{S}'} (\alpha + H(ID'_i)) - \prod_{ID'_i \in S'} H(ID'_i) \right).$$

Then the data user computes $g^\delta = C'_0/F(D') = g^\delta \cdot F(e(g, \mu)^{s'})/F(e(g, \mu)^s)$. It follows that

$$M = C'_3 \cdot e(g^\delta, C'_4) = M/e(v^\delta, g^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}) \cdot e(g^\delta, v^{s \cdot \prod_{ID_i \in S} (\alpha + H(ID_i))}).$$

□

We next prove the security of the PBAA scheme. The security ensures that if a data owner employs the PBAA scheme to encrypt data, then only the users designated by the data owner can access the data. The following theorem guarantees the IND-sSet-CPA security of PBAA as defined by Definition 1.

Theorem 2 (Security). Suppose that the DBDH assumption holds in bilinear groups. Since the IBBE scheme [17] is secure in the random oracle model, the PBAA scheme is IND-sSet-CPA secure in the random oracle model. Specifically, neither CSP nor data users without having correct private keys can correctly decrypt original ciphertexts or re-encrypted ciphertexts.

Proof: We note that the PBAA scheme is built on the IBBE scheme [17]. The (original) ciphertext in PBAA contains an IBBE ciphertext, i.e., (C_0, C_1, C_2) , and the other components $(C_3, \{C_{4,j}, C_{5,j}\})$ of the original ciphertext can be simulated from the IBBE ciphertext. Moreover, the private key of PBAA is similar with that of the IBBE scheme, and the delegation-key generation does not require the master secret key but only the private key. Therefore, we can leverage the adversary's successful attack against the PBAA scheme to break the security of the IBBE scheme. Since the security of the IBBE scheme has been proved by Theorem 1 in [17], there is no PPT adversary that can break the IND-sSet-CPA security of the PBAA scheme. The formal proof is given in the supplemental materials. □

7 COMPARISON AND ANALYSIS

7.1 Functionality Comparison

We compare the PBAA scheme with the related techniques in Table 1. In [12], the authors proposed the first identity-based conditional proxy re-encryption scheme for data sharing. Compared with the PBAA scheme, the scheme in [12] does not support multi-recipient data sharing and the delegation mechanism is not very flexible as only one condition can be specified in the delegation key. The schemes proposed in [32] and [33] address the issue of limited flexibility in delegation and achieve the policy-based delegation similar with ours. Unfortunately, these two schemes do not support multi-recipient data sharing and the scheme in [32] additionally suffers the public-key management problem.

Xu *et al.* [9] proposed a conditional identity-based broadcast PRE (CIBPRE) scheme where a data owner can choose a subset of ciphertexts to share by specifying a condition; besides, the CIBPRE scheme achieves the multi-time re-encryption functionality in that a re-encrypted ciphertext can be further re-encrypted to a new set of receivers. Compared with the PBAA scheme, however, the fine-grained delegation is not achieved in CIBPRE, that is, data owners are not able to formulate access policies to specify which ciphertexts needed to be shared. In [26], the authors proposed an identity-based broadcast PRE scheme that allows a data owner to share encrypted data with a new group of users and the ciphertexts in the scheme can be re-encrypted for multiple times. However, the scheme in [26] does not provide conditional delegation; thus the data owner has to share all of his data at a time, which would incur some privacy issues in practice.

TABLE 1: Comparison with related techniques

Functionality	Ours	[12]	[32]	[9]	[33]	[26]
Identity-based encryption	✓	✓	×	✓	✓	✓
Conditional re-encryption	✓	✓	✓	✓	✓	×
Fine-grained delegation	✓	×	✓	×	✓	×
Multi-recipient data sharing	✓	×	×	✓	×	✓
Multi-time re-encryption	✓	×	×	✓	×	✓

TABLE 2: Computational cost of each algorithm of PBAA

Algorithm	Computations	Entity
Setup	$3N \cdot \tau_e + 1 \cdot \tau_p$	CA
Register	$1 \cdot \tau_e$	CA
Encrypt	$[(3(n+1) + 3m + 1) \cdot \tau_e + 1 \cdot \tau_p]$	Data owner
DKGen	$[(n' + 1) + 6l + 3] \cdot \tau_e + 1 \cdot \tau_p$	Data owner
ReEnc	$n \cdot \tau_e + (3 K + 1) \cdot \tau_p$	CSP
Dec1	$(n - 1) \cdot \tau_e + 2 \cdot \tau_p$	Data user
Dec2	$(n' - 1) \cdot \tau_e + 3 \cdot \tau_p$	Data user

7.2 Theoretical Analysis

We analyze the complexity of the PBAA scheme in terms of computational cost of every algorithm at each entity side, which is summarized in Table 2. We focus on the most time-consuming operations in the bilinear groups such as exponentiations and bilinear pairings. We ignore the computation costs of the hash functions as they are much less than the costs of exponentiations and bilinear pairings. In Table 2, we let τ_e and τ_p denote the execution time of a single exponentiation in $\mathbb{G}$ and one bilinear pairing, respectively. Let N be the maximum size of receiver set, n be the number of users who can decrypt an original ciphertext, m be the number of conditions associated with an original ciphertext, n' be the number of users who can decrypt a re-encrypted ciphertext, l be the number of conditions involved in the access policy of a delegation key, and $|K|$ be the size of the minimum set of conditions that satisfies the access policy.

In the system setup phase, CA takes $3N$ exponentiations and one bilinear pairing to generate system public key. It is worth noting that the computation cost is independent of the total number of conditions in the system. The Register algorithm is very efficient in that it takes CA only one exponentiation to generate a private key. In the Encrypt algorithm, since m conditions are embedded in the ciphertext, the computation cost is linear to the number m of conditions. To authorize a group of n' new users to access encrypted data, the data owner generates a delegation key with an access policy defined over l conditions. Thus, the computation cost of the DKGen algorithm is linear to both the number n' of new receivers and the number l of conditions.

Upon receiving the delegation key, CSP re-encrypts the original ciphertext whose condition set satisfies the access policy of the delegation key. For an original ciphertext, CSP executes $3|K|$ bilinear pairings and n exponentiations to conduct the re-encryption. Thus, the computation cost of the ReEnc algorithm is linear to both the number $|K|$ of conditions and the number n of initial receivers. The computation costs of Dec1 and Dec2 algorithms are linear to the number of receivers but independent of the number of conditions.

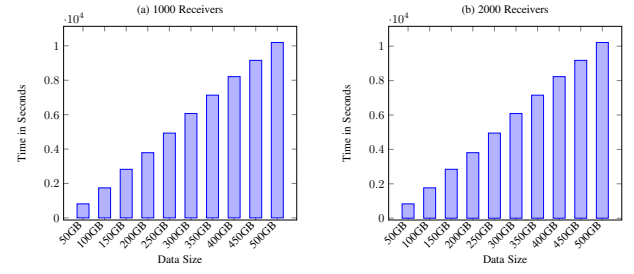


Fig. 4: Execution time of Encrypt

7.3 Experimental Analysis

We conducted a series of experiments to evaluate the performance of the PBAA scheme. The scheme was implemented in a client/server model: the client was built on a Windows 10 PC with a 3.0 GHz Intel Core i7-9700 CPU and 16 GB RAM, and the server was built on a 64-bit dual-core 16GB-memory Windows server in Alibaba Cloud ECS (<https://www.alibabacloud.com/product/ecs>). The client and the server communicate with each other through Socket and UDP protocols. We note that the PBAA scheme is constructed from the bilinear groups, and the PBC library (<https://crypto.stanford.edu/pbc/>) provides routines of basic group operations such as group generation, exponentiation, bilinear map, and hash map. Hence, we utilized the PBC package (written in C) and Visual C++ 6.0 compiler to implement each algorithm of PBAA. In particular, we run the system setup, registration, encryption, delegation and decryption algorithms on the PC client, and run the re-encryption algorithm on the cloud server. In the simulation, we chose an elliptic curve with 160-bit group order, which provides a competitive security level with 1024-bit RSA.

In the implementation, we also followed the idea of key encapsulation for backwards compatibility. Thus, we used 128-bit AES keys to encrypt real data and then encrypted the AES keys with the PBAA's encryption algorithm. The data set used in our experiments is chosen from the medical images of Edinburgh Dermofit Library (<https://licensing.eri.ed.ac.uk/i/software/dermofit-image-library.html>). Besides, since the PBAA scheme allows to share data with multiple recipients by specifying multiple conditions, our experiments were conducted with varying number of recipients and varying number of conditions.

Fig. 4 shows the time cost by the encryption algorithm of PBAA. We chose different size of data to encrypt in this experiment, and fixed the number of conditions (keywords) as $m = 100$ since it seems reasonable to apply no more than 100 keywords to describe a file in practice. From Fig. 4, we can see that the encryption time is linear to the size of data to be encrypted; and for the same data size, the encryption to 2000 receivers consumed (about 20 seconds) more time than the encryption to 1000 receivers.

Fig. 5 shows the time cost by the delegation algorithm of PBAA. Since a data owner can specify an access policy for a delegation key, we conducted this experiment with an access policy of the form $(\text{Cond}_1 \text{ AND } \text{Cond}_2 \cdots \text{AND } \text{Cond}_l)$, where l is set as 100. From Fig. 5, we can see that the delegation time is independent of the size of data, but grows as the number of receivers increases.

Fig. 6 shows the time cost by the re-encryption algorithm of PBAA. It can be seen that the re-encryption time is independent of

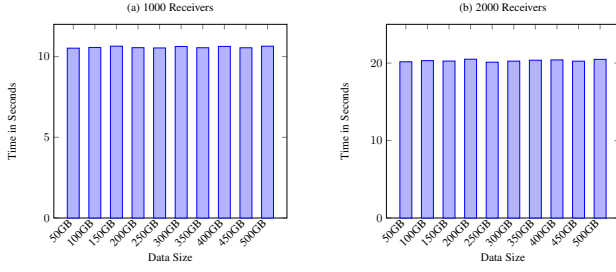


Fig. 5: Execution time of DKGen

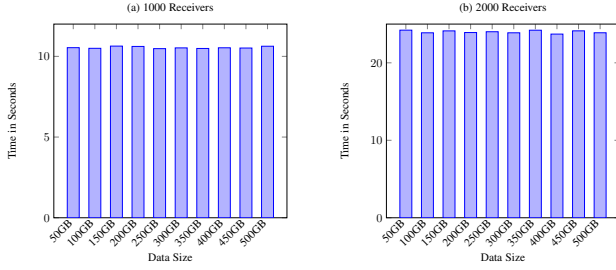


Fig. 6: Execution time of ReEnc

the data size since the cloud does not need to process the encrypted data but only re-encrypts the ciphertext of the encapsulation key (i.e., AES key). As the number of recipients increases, the time cost by the re-encryption algorithm grows.

Fig. 7 shows the times cost by the Dec1 and Dec2 algorithms of PBAA. It can be seen that the times cost by these two algorithms are both linear to the size of data to be decrypted; and as the number of receivers of a ciphertext increases by 1000, the decryption algorithms take an additional 15 seconds. Besides, it is worth noting that Dec1 and Dec2 take the similar time cost for the same size of data, which means that a user would take no extra cost in decrypting a re-encrypted ciphertext compared to decrypting the original ciphertext.

To showcase the flexibility and efficiency of the PBAA scheme, we also implemented the CIBPRE scheme [9] on the same platforms and compared its performance with ours. The CIBPRE scheme supports multi-recipient and conditional data sharing, but only allows one condition to be specified in a ciphertext and a delegation key. Then, in order to achieve the multi-condition based data sharing as PBAA, the CIBPRE scheme has to cut the message into m shares (assuming that there are m conditions to be attached to the message) and then runs the encryption algorithm for each share with a distinct condition. Accordingly, the delegation, re-encryption and decryption algorithms need to be run m times

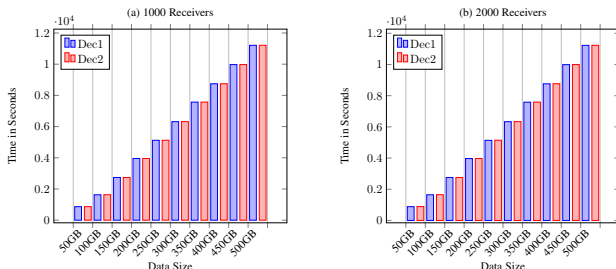


Fig. 7: Execution time of Dec1 and Dec2

repeatedly to fulfill the task of multi-condition based data sharing. In contrast, the PBAA scheme allows to encrypt a message with m conditions (i.e., $\{\text{Cond}_1, \text{Cond}_2, \dots, \text{Cond}_m\}$) at a time and the delegation algorithm can be run only once with the access policy ($\text{Cond}_1 \text{ AND } \text{Cond}_2 \dots \text{ AND } \text{Cond}_m$) to share the message.

Fig. 8, Fig. 9, Fig. 10 and Fig. 11 show the execution times of the encryption, delegation, re-encryption and decryption algorithms of the PBAA and CIBPRE schemes. Suppose that there are 1000 receivers specified to decrypt a ciphertext in both the schemes and 1GB data is chosen for encryption. We conducted this comparative experiment with different number of conditions, varying from 100 to 1000 with interval 100. From Fig. 8, Fig. 9, Fig. 10 and Fig. 11, it can be seen that the times cost by the CIBPRE scheme are dramatically higher than those of our PBAA scheme. This is because that the parameter m (the number of conditions) is a multiplicative factor of the time consumed by each algorithm of CIBPRE, but an additive factor of the time consumed by the encryption, delegation and re-encryption algorithms of PBAA (the time cost by the decryption is independent of m). The above experimental analyses reveal that the PBAA scheme achieves a practicable access authorization in that data owners can efficiently and flexibly share their encrypted data with a group of users and each user can access the data at a very low cost.

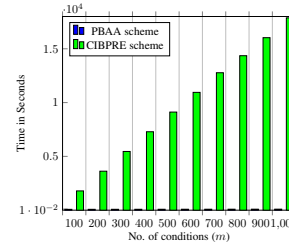


Fig. 8: Encryption

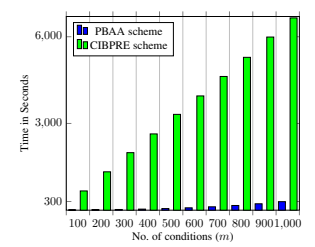


Fig. 9: Delegation

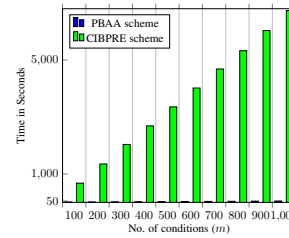


Fig. 10: Re-encryption

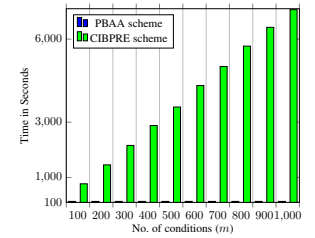


Fig. 11: Decryption-2

8 CONCLUSION

In this paper, we investigated how to flexibly share encrypted data in the cloud. The proposed PBAA scheme utilizes IBBE to achieve multi-recipient data sharing, and also realizes a fine-grained re-encryption mechanism by adopting linear secret sharing technique. Then, for the first time, the PBAA scheme provides a flexible access authorization for data owners to share their encrypted data. In particular, the data owner can generate a delegation key with an access policy and then send this key to the cloud such that it can re-encrypt the encrypted data satisfying the access policy to be accessible to a new group of recipients. The proposed scheme is provably secure and theoretical and experimental analyses reveal the efficiency and practicality of the scheme.

ACKNOWLEDGMENT

This paper is supported in part by the National Key R&D program of China through projects 2018YFB0704000, 2020YFB10056, 2019QY(Y)0602, 2017YFB1400700 and 2017YFB0802500, in part by the National Natural Science Foundation of China through projects 61902123, U20A20174, 62002106, 62002112, 61932011, 61972019, 61772191, 61772538, 61532021, 91646203 and 61672083, in part by the Science and Technology Key Projects of Hunan Province through projects 2015TP1004, 2018TP2023 and 2019WK2072, in part by the China Postdoctoral Science Foundation through projects 2019M662769 and 2020M672488, in part by the Natural Science Foundation of Hunan Province through project 2020JJ5085, in part by the Science and Technology Projects of Changsha City through projects kq2004025 and kq2004027, and in part by Hangzhou Innovation Institute, Beihang University, through project 2020-Y9-A-024.

REFERENCES

- [1] Y. Wang, Y. Ding, Q. Wu, Y. Wei, B. Qin, and H. Wang, "Privacy-preserving cloud-based road condition monitoring with source authentication in vanets," *IEEE Trans. Information Forensics and Security*, vol. 14, no. 7, pp. 1779–1790, 2019.
- [2] H. Yin, Z. Qin, J. Zhang, L. Ou, F. Li, and K. Li, "Secure conjunctive multi-keyword ranked search over encrypted cloud data for multiple data owners," *Future Generation Computer Systems*, vol. 100, pp. 689–700, 2019.
- [3] <http://www.healthvault.com>.
- [4] <https://www.google.com/health>.
- [5] M. Blaze, G. Bleumer, and M. Strauss, "Divertible protocols and atomic proxy cryptography," in *EUROCRYPT 1998*. Springer Berlin Heidelberg, 1998, pp. 127–144.
- [6] M. Green and G. Ateniese, "Identity-based proxy re-encryption," in *ACNS 2007*. Springer Berlin Heidelberg, 2007, pp. 288–306.
- [7] H. Deng, Z. Qin, Q. Wu, Z. Guan, R. H. Deng, Y. Wang, and Y. Zhou, "Identity-based encryption transformation for flexible sharing of encrypted data in public cloud," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3168–3180, 2020.
- [8] S. Yin, H. Li, and L. Teng, "A novel proxy re-encryption scheme based on identity property and stateless broadcast encryption under cloud environment," *International Journal of Network Security*, vol. 21, no. 5, pp. 797–803, 2019.
- [9] P. Xu, T. Jiao, Q. Wu, W. Wang, and H. Jin, "Conditional identity-based broadcast proxy re-encryption and its application to cloud email," *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 66–79, 2015.
- [10] Q. Huang, Y. Yang, and J. Fu, "Precise: Identity-based private data sharing with conditional proxy re-encryption in online social networks," *Future Generation Computer Systems*, vol. 86, pp. 1523–1533, 2018.
- [11] C. Ge, L. Zhou, J. Xia, P. Szalachowski, and C. Su, "A secure fine-grained identity-based proxy broadcast re-encryption scheme for micro-video subscribing system in clouds," in *International Symposium on Security and Privacy in Social Networks and Big Data*. Springer, 2019, pp. 139–151.
- [12] J. Shao, G. Wei, Y. Ling, and M. Xie, "Identity-based conditional proxy re-encryption," in *2011 IEEE International Conference on Communications (ICC)*. IEEE, 2011, pp. 1–5.
- [13] A. Fiat and M. Naor, "Broadcast encryption," in *Annual International Cryptology Conference*. Springer, 1993, pp. 480–491.
- [14] D. Boneh, C. Gentry, and B. Waters, "Collusion resistant broadcast encryption with short ciphertexts and private keys," in *CRYPTO 2005*. Springer, 2005, pp. 258–275.
- [15] D. Boneh and B. Waters, "A fully collusion resistant broadcast, trace, and revoke system," in *Proceedings of the 13th ACM conference on Computer and communications security*. ACM, 2006, pp. 211–220.
- [16] J. Kim, W. Susilo, M. H. Au, and J. Seberry, "Adaptively secure identity-based broadcast encryption with a constant-sized ciphertext," *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 3, pp. 679–693, 2015.
- [17] C. Delerablée, "Identity-based broadcast encryption with constant size ciphertexts and private keys," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2007, pp. 200–215.
- [18] R. Sakai and J. Furukawa, "Identity-based broadcast encryption," *IACR Cryptology ePrint Archive*, vol. 2007, p. 217, 2007.
- [19] J. Lai, Y. Mu, F. Guo, W. Susilo, and R. Chen, "Anonymous identity-based broadcast encryption with revocation for file sharing," in *Australasian Conference on Information Security and Privacy*. Springer, 2016, pp. 223–239.
- [20] —, "Fully privacy-preserving and revocable id-based broadcast encryption for data access control in smart city," *Personal and Ubiquitous Computing*, vol. 21, no. 5, pp. 855–868, 2017.
- [21] B. Waters, "Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization," in *International Workshop on Public Key Cryptography*. Springer, 2011, pp. 53–70.
- [22] Y. Rouselakis and B. Waters, "Practical constructions and new proof methods for large universe attribute-based encryption," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. ACM, 2013, pp. 463–474.
- [23] L. Zhang, H. Xiong, Q. Huang, J. Li, K.-K. R. Choo, and J. Li, "Cryptographic solutions for cloud storage: Challenges and research opportunities," *IEEE Transactions on Services Computing*, 2019.
- [24] L. Zhang, H. Ma, Z. Liu, and E. Dong, "Security analysis and improvement of a collusion-resistant identity-based proxy re-encryption scheme," in *International Conference on Broadband and Wireless Computing, Communication and Applications*. Springer, 2016, pp. 839–846.
- [25] J. Shao and Z. Cao, "Multi-use unidirectional identity-based proxy re-encryption from hierarchical identity-based encryption," *Information Sciences*, vol. 206, pp. 83–95, 2012.
- [26] C. Ge, Z. Liu, J. Xia, and F. Liming, "Revocable identity-based broadcast proxy re-encryption for data sharing in clouds," *IEEE Transactions on Dependable and Secure Computing*, 2019.
- [27] J. Weng, R. H. Deng, X. Ding, C.-K. Chu, and J. Lai, "Conditional proxy re-encryption secure against chosen-ciphertext attack," in *Proceedings of the 4th International Symposium on Information, Computer, and Communications Security*. ACM, 2009, pp. 322–332.
- [28] S. S. Vivek, S. S. D. Selvi, V. Radhakrishnan, and C. P. Rangan, "Conditional proxy re-encryption—a more efficient construction," in *International Conference on Network Security and Applications*. Springer, 2011, pp. 502–512.
- [29] C.-K. Chu, J. Weng, S. S. Chow, J. Zhou, and R. H. Deng, "Conditional proxy broadcast re-encryption," in *Australasian Conference on Information Security and Privacy*. Springer, 2009, pp. 327–342.
- [30] K. Liang, C.-K. Chu, X. Tan, D. S. Wong, C. Tang, and J. Zhou, "Chosen-ciphertext secure multi-hop identity-based conditional proxy re-encryption with constant-size ciphertexts," *Theoretical Computer Science*, vol. 539, pp. 87–105, 2014.
- [31] L. Fang, J. Wang, C. Ge, and Y. Ren, "Fuzzy conditional proxy re-encryption," *Science China Information Sciences*, vol. 56, no. 5, pp. 1–13, 2013.
- [32] Y. Yang, H. Lu, J. Weng, Y. Zhang, and K. Sakurai, "Fine-grained conditional proxy re-encryption and application," in *International Conference on Provable Security*. Springer, 2014, pp. 206–222.
- [33] C. Ge, W. Susilo, J. Wang, and L. Fang, "Identity-based conditional proxy re-encryption with fine grain policy," *Computer Standards & Interfaces*, vol. 52, pp. 1–9, 2017.
- [34] C. K. Chu and W. G. Tzeng, "Identity-based proxy re-encryption without random oracles," in *ISC 2007*. Springer Berlin Heidelberg, 2007, pp. 189–202.
- [35] H. Deng, Z. Qin, Q. Wu, Z. Guan, and Y. Zhou, "Flexible attribute-based proxy re-encryption for efficient data sharing," *Information Sciences*, vol. 511, pp. 94–113, 2020.
- [36] D. Boneh and X. Boyen, "Efficient selective-id secure identity-based encryption without random oracles," in *EUROCRYPT 2004*. Springer Berlin Heidelberg, 2004, pp. 223–238.
- [37] A. Beigel, *Secure schemes for secret sharing and key distribution*. Technion-Israel Institute of technology, Faculty of computer science, 1996.
- [38] S. Wang, K. Liang, J. K. Liu, J. Chen, J. Yu, and W. Xie, "Attribute-based data sharing scheme revisited in cloud computing," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 8, pp. 1661–1673, 2016.
- [39] S. K. Sood, A. K. Sarje, and K. Singh, "A secure dynamic identity based authentication protocol for multi-server architecture," *Journal of Network and Computer Applications*, vol. 34, no. 2, pp. 609–618, 2011.
- [40] R. Oppliger, R. Hauser, and D. Basin, "Ssl/tls session-aware user authentication," *Computer*, vol. 41, no. 3, pp. 59–65, 2008.
- [41] H. Li, Y. Dai, L. Tian, and H. Yang, "Identity-based authentication for cloud computing," in *IEEE international conference on cloud computing*. Springer, 2009, pp. 157–166.

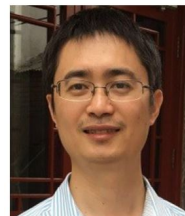
- [42] D. He, S. Zeadally, B. Xu, and X. Huang, "An efficient identity-based conditional privacy-preserving authentication scheme for vehicular ad hoc networks," *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 12, pp. 2681–2691, 2015.
- [43] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2001, pp. 514–532.
- [44] D. Clarke, S. Devadas, M. Van Dijk, B. Gassend, and G. E. Suh, "Incremental multiset hash functions and their application to memory integrity checking," in *International conference on the theory and application of cryptography and information security*. Springer, 2003, pp. 188–207.
- [45] S. Chandrasekhar, S. Chakrabarti, and M. Singhal, "A trapdoor hash-based mechanism for stream authentication," *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 5, pp. 699–713, 2012.
- [46] A. Shamir and Y. Tauman, "Improved online/offline signature schemes," in *Annual International Cryptology Conference*. Springer, 2001, pp. 355–367.
- [47] A. A. Yavuz, "Immutable authentication and integrity schemes for outsourced databases," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 1, pp. 69–82, 2016.



Hua Deng received his M.S. degree in cryptography from Southwest Jiaotong University, China, in 2010 and Ph.D. degree in information security from Wuhan University, China, in 2015. Currently he is an associate research fellow with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include applied cryptography, data security and privacy, cloud security.



Qianhong Wu received the M.Sc. degree in applied mathematics from Sichuan University, Sichuan, China, and the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2001 and 2004, respectively. Since then, he has been an Associate Research Fellow with the University of Wollongong, Wollongong, Australia, an Associate Professor with Wuhan University, Wuhan, China, and a Senior Researcher with the Universitat Rovira i Virgili, Tarragona, Catalonia. His research interests include cryptography, information security and privacy, and ad hoc network security. He has been a main researcher or project holder/coholder for more than ten Chinese-, Australian-, and Spanish-funded projects. He has authored over 60 publications and served on the program committees of several international conferences on information security and privacy. He is a member of the International Association for Cryptologic Research.



Hui Yin received his Ph.D. degree from the College of Information Science and Engineering at Hunan University, China, in 2018. He is currently an associate professor at the College of Applied Mathematics and Computer Engineering, Changsha University, China. His interests are information security, privacy protection, and applied cryptography.



Jixin Zhang received the M.S. degree in computer science and technology from Wuhan University of Technology and received the Ph.D. degree in computer science and technology from Hunan University, in 2014 and 2019, respectively. Currently, he is an assistant professor of Computer Science in Hubei University of Technology. His primary researches focus on Artificial intelligence and security.



Aniello Castiglione (S'04–M'08) received the Ph.D. degree in computer science from the University of Salerno, Fisciano, Italy. He is an Adjunct Professor with the University of Salerno, and the University of Naples Federico II, Naples, Italy. He has authored or co-authored over 190 papers in international journals and conferences. His current research interests include information forensics, digital forensics, security and privacy on cloud, communication networks, applied cryptography, and sustainable computing. Dr. Castiglione has served in the organization (mainly as the Program Chair and a TPC member) of approximately 200 international conferences. He served as a Reviewer for approximately 100 international journals and the Managing Editor of two ISI-ranked international journals. He was a Guest Editor of approximately 20 special issues and served as an Editor on approximately 10 Editorial Boards of international journals. One of his papers (published in the IEEE Transactions on Dependable and Secure Computing) was selected as the "Featured Article" in the "IEEE Cybersecurity Initiative," in 2014, while in 2018 another paper (published in the IEEE Transactions on Cloud Computing) was selected as the "Featured Article" in the "IEEE Cloud Computing Initiative".



Zheng Qin received the Ph.D. degree in computer software and theory from Chongqing University, China, in 2001. From 2010 to 2011, he served as a Visiting Scholar with the Department of Computer Science, Michigan University. He is currently a Professor with the College of Computer Science and Electronic Engineering, Hunan University, where he also serves as the Vice Dean. His main interests are network and data security, privacy, data analytics and applications, machine learning, and applied cryptography.